

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего
образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И
УПРАВЛЕНИЯ

ДОМАШНЕЕ ЗАДАНИЕ №2

«Разработка консольного приложения с применением ООП»

ДИСЦИПЛИНА: «Программирование на основе классов и шаблонов»

Вариант 24: операторы связи - тарифы - price_month

Выполнил: студент гр. ИУ5-22Б _____

(Подпись) (Смыков В.А.)

Проверил: _____

(Подпись) (Гапанюк Ю.Е.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

Москва, 2026 г

1. Цель работы

Целью работы является разработка консольного приложения на C# с применением объектно-ориентированного подхода для хранения данных в SQLite, выполнения CRUD-операций и формирования текстовых отчётов через Fluent Interface.

2. Вариант задания

Выполняется вариант 24. Предметная область: операторы связи и тарифы. Справочная таблица хранит операторов связи, основная таблица хранит тарифы, а числовое поле `price_month` задаёт абонентскую плату в рублях в месяц.

№	Справочник	Основная таблица	Числовое поле	Расшифровка
24	Операторы связи	Тарифы	<code>price_month</code>	Абонентская плата (руб./мес.)

Связь между таблицами относится к типу «один ко многим»: один оператор связи может иметь несколько тарифов, но каждый тариф относится только к одному оператору связи.

3. Теоретическая часть

SQLite используется как встроенная реляционная СУБД. Программа создаёт две таблицы: `operators` и `tariffs`. Таблица `tariffs` содержит внешний ключ `operator_id`, который ссылается на `operators.operator_id`.

Класс `ReportBuilder` реализует `Fluent Interface`: промежуточные методы `Query`, `Title`, `Header`, `ColumnWidths`, `Numbered` и `Footer` возвращают `this` и позволяют записывать настройку отчёта в виде цепочки вызовов. Терминальные методы `Build`, `Print` и `SaveToFile` завершают построение отчёта.

4. Метаграфовое ситуационное описание и блок-схемы

4.1. Модель данных

На схеме показаны два класса-модели. `TelecomOperator` является стороной «один», `Tariff` является стороной «много». Поле `Tariff.OperatorId` является внешним ключом и указывает на `TelecomOperator.Id`. Для `PriceMonth` задано ограничение ≥ 0 .

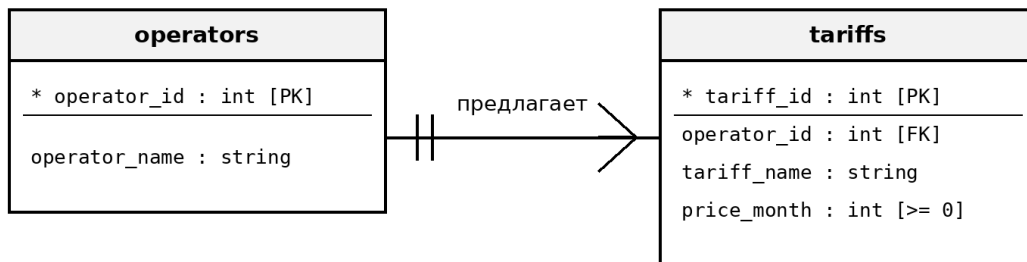
Схема А. Нотация Чена: операторы связи - тарифы



Один оператор связи может предлагать несколько тарифов; каждый тариф относится к одному оператору.

Рис. 1. Модель данных в нотации Чена

Схема А. Entity Relationship Diagram



PK - первичный ключ; FK - внешний ключ tariffs.operator_id -> operators.operator_id.

Рис. 2. Модель данных в нотации «воронья лапка»

4.2. Схема работы ReportBuilder

Схема Б. Работа ReportBuilder (Fluent Interface)

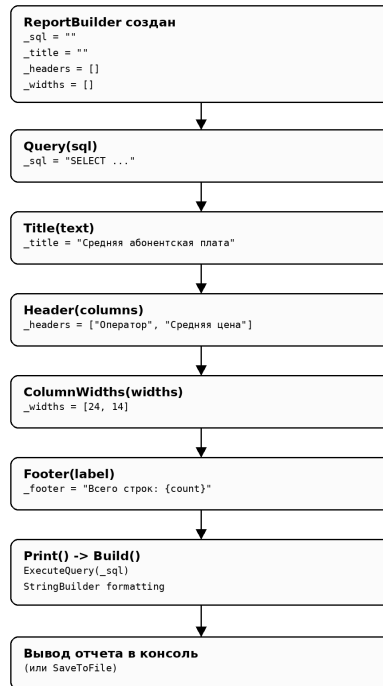


Рис. 3. Блок-схема работы цепочки ReportBuilder

4.3. Блок-схема консольного приложения

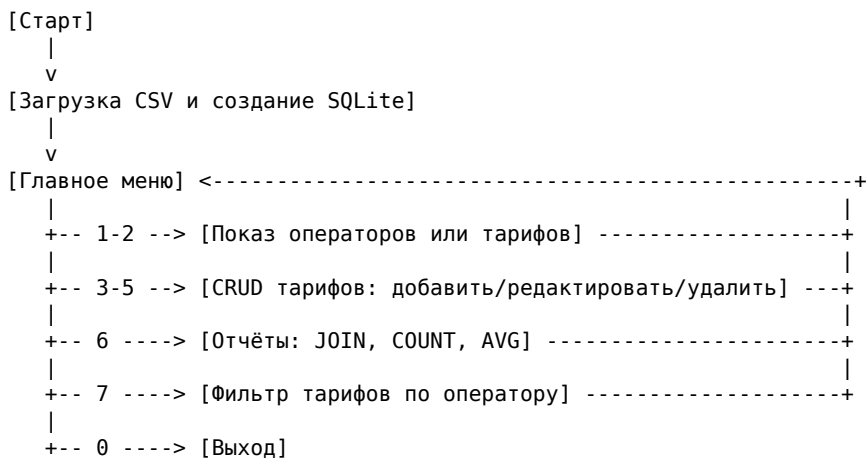


Рис. 4. Общая блок-схема консольного приложения

5. Структура проекта

Домашнее задание размещено в папке 07_Homework2_OopSqlite. По просьбе преподавателя исходный код объединён в один файл Program.cs. Внутри этого файла находятся точка входа и все классы приложения: Program, TelecomOperator, Tariff, QueryResult, DatabaseManager и ReportBuilder.

Файл	Назначение
Program.cs	Единый файл исходного кода: меню, модели, DatabaseManager, ReportBuilder и QueryResult.
operators.csv	Тестовые данные справочника операторов связи.
tariffs.csv	Тестовые данные основной таблицы тарифов.
Homework2_OopSqlite.csproj	Файл проекта .NET 8.0 с подключением Microsoft.Data.Sqlite.
diagrams/*.puml	PlantUML-исходники схем.
Отчет_Д32_Смыков_ВА.pdf	Данный отчёт со схемами и описанием решения.

6. CSV-файлы с тестовыми данными

Созданы два CSV-файла с разделителем «;» и заголовками в первой строке. В справочнике содержится 4 записи, а в основной таблице - 12 записей.

operators.csv

```
operator_id;operator_name
1;МТС
2;Билайн
3;МегаФон
4;Tele2
5;Yota
```

tariffs.csv

```
tariff_id;operator_id;tariff_name;price_month
1;1;МТС Smart;650
2;1;МТС Тарифище;890
3;1;МТС Супер;450
4;2;Билайн Близкие люди 1;700
5;2;Билайн Аппер;1100
6;2;Билайн Первые гига;550
7;3;МегаФон Включайся;750
8;3;МегаФон Максимум;1250
9;3;МегаФон Минимум;500
10;4;Tele2 Мой онлайн;600
11;4;Tele2 Black;800
12;4;Tele2 Везде онлайн;450
13;5;Yota Интернет;720
14;5;Yota Безлимит;990
15;5;Yota Базовый;390
```

7. Реализация программы

В программе реализованы классы TelecomOperator и Tariff. В классе Tariff свойство PriceMonth содержит валидацию: при попытке записать отрицательное значение генерируется ArgumentException.

DatabaseManager создаёт таблицы operators и tariffs, импортирует данные из CSV-файлов, возвращает списки записей, добавляет, редактирует и удаляет тарифы. Для безопасного выполнения SQL используются параметры запроса.

Главное меню работает в цикле. Для добавления тарифа сначала выводится список операторов связи, затем пользователь выбирает внешний ключ. При редактировании текущие значения показываются заранее, а Enter без ввода оставляет поле без изменений.

8. Обязательные отчёты

Через пункт «Отчёты» доступны три обязательных отчёта.

- Полный список тарифов с названиями операторов связи: используется JOIN и сортировка по названию тарифа.
- Количество тарифов у каждого оператора связи: используется GROUP BY и COUNT(*).
- Средняя абонентская плата по операторам связи: используется GROUP BY и AVG(price_month).

Пример цепочки вызовов для первого отчёта:

```
new ReportBuilder(db)
  .Query(@"
    SELECT t.tariff_name, o.operator_name, t.price_month
    FROM tariffs t
    JOIN operators o ON t.operator_id = o.operator_id
    ORDER BY t.tariff_name;")
  .Title("Тарифы по операторам связи")
  .Header("Тариф", "Оператор", "Цена/мес.")
  .ColumnWidths(5, 28, 18, 12)
  .Numbered()
  .Footer("Всего тарифов: {count}")
  .Print();
```

9. Результаты выполнения

После запуска пользователь видит главное меню приложения:

```
=== ДОМАШНЕЕ ЗАДАНИЕ №2. ВАРИАНТ 24 ===
Автор: Смыков В.А.
Предметная область: операторы связи и тарифы.
```

```
=== Управление данными ===
1 - Показать всех операторов связи
2 - Показать все тарифы
3 - Добавить тариф
4 - Редактировать тариф
5 - Удалить тариф
6 - Отчеты
7 - Фильтр по оператору связи
0 - Выход
Ваш выбор:
```

Пример результата отчёта 1:

```
=== Тарифы по операторам связи ===
```

№	Тариф	Оператор	Цена/мес.
1	Black	Tele2	950
2	VIP	МегаФон	1800
3	Без переплат Минимум	МегаФон	550
4	Билайн Всё за 700	Билайн	700
5	Близкие люди 2	Билайн	850
6	Везде онлайн	Tele2	750
7	Интернет PRO	Билайн	990
8	МТС Premium	МТС	1590
9	МТС Smart	МТС	650
10	МегаФон Максимум	МегаФон	900
11	Мой онлайн	Tele2	600

12 Тарифище МТС 790

Всего тарифов: 12

Пример результата отчёта 2:

=== Количество тарифов у операторов ===

№	Оператор	Количество
1	Tele2	3
2	Билайн	3
3	МТС	3
4	МегаФон	3

Всего операторов: 4

Пример результата отчёта 3:

=== Средняя абонентская плата по операторам ===

№	Оператор	Средняя цена
1	МегаФон	1083.33
2	МТС	1010.0
3	Билайн	846.67
4	Tele2	766.67

Всего операторов: 4

Фильтр по оператору позволяет выбрать запись справочника и вывести только тарифы выбранного оператора связи. Например, для оператора #2 программа выводит тарифы «Билайн Всё за 700», «Близкие люди 2» и «Интернет PRO».

10. Порядок запуска

Для запуска необходимо установить .NET SDK 8.0. На Windows перед запуском рекомендуется выполнить `chcp 65001`, чтобы консоль корректно отображала кириллицу.

```
cd 07_Homework2_OopSqlite
dotnet restore
dotnet run
```

Из корня архива домашнее задание можно запустить командой:

```
dotnet run --project 07_Homework2_OopSqlite/Homework2_OopSqlite.csproj
```

11. Вывод

В ходе выполнения работы было разработано консольное приложение на C# для предметной области варианта 24 «Операторы связи - Тарифы». Реализованы CSV-файлы с тестовыми данными, классы-модели с XML-комментариями и валидацией, класс `DatabaseManager` для работы с `SQLite`, интерактивное меню и класс `ReportBuilder` с `Fluent Interface`. Программа формирует три обязательных отчёта с использованием `JOIN`, `COUNT` и `AVG`, а также поддерживает дополнительные возможности: нумерацию строк, итоговую строку, сохранение отчёта в файл и фильтрацию тарифов по оператору связи. Исходный код ДЗ2 объединён в один файл `Program.cs`.

Приложение А. Исходный код Program.cs

Ниже приведён полный единый файл исходного кода домашнего задания №2.

```
using System;
using System.Collections.Generic;
using System.Globalization;
using System.IO;
using System.Text;
using Microsoft.Data.Sqlite;

/*
Автор: Смыков В.А.
Домашнее задание №2. Вариант 24.
Предметная область: операторы связи и тарифы.
Числовое поле: price_month – абонентская плата, руб./мес.

Все классы домашнего задания объединены в одном файле Program.cs:
Program, TelecomOperator, Tariff, QueryResult, DatabaseManager, ReportBuilder.

Запуск из папки проекта:
dotnet restore
dotnet run
*/

// =====
// Program.cs
// =====

internal static class Program
{
    private static void Main()
    {
        Console.InputEncoding = Encoding.UTF8;
        Console.OutputEncoding = Encoding.UTF8;
        CultureInfo.CurrentCulture = CultureInfo.InvariantCulture;
        CultureInfo.CurrentUICulture = CultureInfo.InvariantCulture;

        string baseDir = AppContext.BaseDirectory;
        string dbPath = Path.Combine(baseDir, "telecom_tariffs.db");
        string operatorsCsvPath = Path.Combine(baseDir, "operators.csv");
        string tariffsCsvPath = Path.Combine(baseDir, "tariffs.csv");

        EnsureCsvFiles(operatorsCsvPath, tariffsCsvPath);

        DatabaseManager db = new(dbPath);
        db.ImportFromCsv(operatorsCsvPath, tariffsCsvPath);

        Console.WriteLine("=== ДОМАШНЕЕ ЗАДАНИЕ №2. ВАРИАНТ 24 ===");
        Console.WriteLine("Автор: Смыков В.А.");
        Console.WriteLine("Предметная область: операторы связи и тарифы.");
        Console.WriteLine();

        RunMenu(db);
    }

    private static void RunMenu(DatabaseManager db)
    {
        while (true)
        {
            Console.WriteLine();
            Console.WriteLine("=== Управление данными ===");
            Console.WriteLine("1 – Показать всех операторов связи");
            Console.WriteLine("2 – Показать все тарифы");
            Console.WriteLine("3 – Добавить тариф");
            Console.WriteLine("4 – Редактировать тариф");
            Console.WriteLine("5 – Удалить тариф");
            Console.WriteLine("6 – Отчеты");
            Console.WriteLine("7 – Фильтр по оператору связи");
            Console.WriteLine("0 – Выход");
            Console.Write("Ваш выбор: ");

            string? choice = Console.ReadLine();
            Console.WriteLine();

            try
            {
                switch (choice)
                {
                    case "1":
                        ShowOperators(db);
                        break;
                    case "2":
                        ShowTariffs(db);
                        break;
                    case "3":
                        AddTariff(db);

```



```

        break;
    case "4":
        EditTariff(db);
        break;
    case "5":
        DeleteTariff(db);
        break;
    case "6":
        ShowReports(db);
        break;
    case "7":
        FilterByOperator(db);
        break;
    case "0":
        Console.WriteLine("Работа программы завершена.");
        return;
    default:
        Console.WriteLine("Ошибка: неизвестный пункт меню.");
        break;
    }
}
catch (Exception ex)
{
    Console.WriteLine("Ошибка: " + ex.Message);
}
}

private static void ShowOperators(DatabaseManager db)
{
    Console.WriteLine("=== Операторы связи ===");
    List<TelecomOperator> operators = db.GetAllOperators();

    for (int i = 0; i < operators.Count; i++)
        Console.WriteLine(operators[i]);
}

private static void ShowTariffs(DatabaseManager db)
{
    Console.WriteLine("=== Тарифы ===");
    List<Tariff> tariffs = db.GetAllTariffs();

    for (int i = 0; i < tariffs.Count; i++)
        Console.WriteLine(tariffs[i]);
}

private static void AddTariff(DatabaseManager db)
{
    Console.WriteLine("=== Добавление тарифа ===");
    ShowOperators(db);

    Console.Write("Введите Id оператора связи: ");
    if (!int.TryParse(Console.ReadLine(), out int operatorId))
    {
        Console.WriteLine("Ошибка: Id должен быть целым числом.");
        return;
    }

    Console.Write("Название тарифа: ");
    string name = Console.ReadLine()?.Trim() ?? string.Empty;
    if (string.IsNullOrEmpty(name))
    {
        Console.WriteLine("Ошибка: название тарифа не может быть пустым.");
        return;
    }

    Console.Write("Абонентская плата, руб./мес.: ");
    if (!int.TryParse(Console.ReadLine(), out int priceMonth))
    {
        Console.WriteLine("Ошибка: абонентская плата должна быть целым числом.");
        return;
    }

    Tariff tariff = new(db.GetNextTariffId(), operatorId, name, priceMonth);
    db.AddTariff(tariff);
    Console.WriteLine("Тариф добавлен.");
}

private static void EditTariff(DatabaseManager db)
{
    Console.WriteLine("=== Редактирование тарифа ===");
    ShowTariffs(db);

    Console.Write("Введите Id тарифа: ");
    if (!int.TryParse(Console.ReadLine(), out int id))
    {
        Console.WriteLine("Ошибка: Id должен быть целым числом.");
        return;
    }

    Tariff? tariff = db.GetTariffById(id);

```

```

        if (tariff is null)
        {
            Console.WriteLine("Тариф с указанным Id не найден.");
            return;
        }

        Console.WriteLine("Текущие значения:");
        Console.WriteLine(tariff);
        Console.WriteLine("Нажмите Enter, если поле не нужно менять.");

        Console.Write($"Название тарифа ({tariff.Name}): ");
        string? nameInput = Console.ReadLine();
        if (!string.IsNullOrEmpty(nameInput))
            tariff.Name = nameInput.Trim();

        ShowOperators(db);
        Console.Write($"Id оператора связи ({tariff.OperatorId}): ");
        string? operatorInput = Console.ReadLine();
        if (!string.IsNullOrEmpty(operatorInput))
        {
            if (!int.TryParse(operatorInput, out int operatorId))
            {
                Console.WriteLine("Ошибка: Id оператора должен быть целым числом.");
                return;
            }

            tariff.OperatorId = operatorId;
        }

        Console.Write($"Абонентская плата ({tariff.PriceMonth}): ");
        string? priceInput = Console.ReadLine();
        if (!string.IsNullOrEmpty(priceInput))
        {
            if (!int.TryParse(priceInput, out int priceMonth))
            {
                Console.WriteLine("Ошибка: абонентская плата должна быть целым числом.");
                return;
            }

            tariff.PriceMonth = priceMonth;
        }

        db.UpdateTariff(tariff);
        Console.WriteLine("Тариф обновлен.");
    }

    private static void DeleteTariff(DatabaseManager db)
    {
        Console.WriteLine("=== Удаление тарифа ===");
        ShowTariffs(db);

        Console.Write("Введите Id тарифа для удаления: ");
        if (!int.TryParse(Console.ReadLine(), out int id))
        {
            Console.WriteLine("Ошибка: Id должен быть целым числом.");
            return;
        }

        db.DeleteTariff(id);
        Console.WriteLine("Тариф удален, если запись с таким Id существовала.");
    }

    private static void ShowReports(DatabaseManager db)
    {
        Console.WriteLine("=== Отчеты ===");

        new ReportBuilder(db)
            .Query(@"
                SELECT t.tariff_name, o.operator_name, t.price_month
                FROM tariffs t
                JOIN operators o ON t.operator_id = o.operator_id
                ORDER BY t.tariff_name;")
            .Title("Тарифы по операторам связи")
            .Header("Тариф", "Оператор", "Цена/мес.")
            .ColumnWidths(5, 28, 18, 12)
            .Numbered()
            .Footer("Всего тарифов: {count}")
            .Print();

        new ReportBuilder(db)
            .Query(@"
                SELECT o.operator_name, COUNT(*) AS tariff_count
                FROM tariffs t
                JOIN operators o ON t.operator_id = o.operator_id
                GROUP BY o.operator_name
                ORDER BY o.operator_name;")
            .Title("Количество тарифов у каждого оператора")
            .Header("Оператор", "Количество")
            .ColumnWidths(24, 12)
            .Footer("Всего операторов в отчете: {count}")
            .Print();
    }

```

```

string avgReportPath = Path.Combine(AppContext.BaseDirectory, "avg_price_report.txt");

new ReportBuilder(db)
    .Query(@"
        SELECT o.operator_name, ROUND(AVG(t.price_month), 2) AS avg_price_month
        FROM tariffs t
        JOIN operators o ON t.operator_id = o.operator_id
        GROUP BY o.operator_name
        ORDER BY avg_price_month DESC;")
    .Title("Средняя абонентская плата по операторам")
    .Header("Оператор", "Средняя цена")
    .ColumnWidths(24, 14)
    .Footer("Всего строк: {count}")
    .Print();

new ReportBuilder(db)
    .Query(@"
        SELECT o.operator_name, ROUND(AVG(t.price_month), 2) AS avg_price_month
        FROM tariffs t
        JOIN operators o ON t.operator_id = o.operator_id
        GROUP BY o.operator_name
        ORDER BY avg_price_month DESC;")
    .Title("Средняя абонентская плата по операторам")
    .Header("Оператор", "Средняя цена")
    .ColumnWidths(24, 14)
    .Footer("Всего строк: {count}")
    .SaveToFile(avgReportPath);

Console.WriteLine($"Отчет со средней ценой дополнительно сохранен в файл: {avgReportPath}");
}

private static void FilterByOperator(DatabaseManager db)
{
    Console.WriteLine("=== Фильтр по оператору связи ===");
    ShowOperators(db);

    Console.WriteLine("Введите Id оператора связи: ");
    if (!int.TryParse(Console.ReadLine(), out int operatorId))
    {
        Console.WriteLine("Ошибка: Id должен быть целым числом.");
        return;
    }

    List<Tariff> tariffs = db.GetTariffsByOperator(operatorId);

    Console.WriteLine($"Тарифы оператора #{operatorId}:");
    for (int i = 0; i < tariffs.Count; i++)
        Console.WriteLine(tariffs[i]);

    Console.WriteLine($"Найдено тарифов: {tariffs.Count}");
}

private static void EnsureCsvFiles(string operatorsCsvPath, string tariffsCsvPath)
{
    if (!File.Exists(operatorsCsvPath))
    {
        File.WriteAllText(operatorsCsvPath,
            "operator_id;operator_name\n" +
            "1;МТС\n" +
            "2;Билайн\n" +
            "3;МегаФон\n" +
            "4;Tele2\n" +
            "5;Yota\n",
            Encoding.UTF8);
    }

    if (!File.Exists(tariffsCsvPath))
    {
        File.WriteAllText(tariffsCsvPath,
            "tariff_id;operator_id;tariff_name;price_month\n" +
            "1;1;МТС Smart;650\n" +
            "2;1;МТС Тарифище;890\n" +
            "3;1;МТС Супер;450\n" +
            "4;2;Билайн Близкие люди 1;700\n" +
            "5;2;Билайн Аппер;1100\n" +
            "6;2;Билайн Первые гига;550\n" +
            "7;3;МегаФон Включайся;750\n" +
            "8;3;МегаФон Максимум;1250\n" +
            "9;3;МегаФон Минимум;500\n" +
            "10;4;Tele2 Мой онлайн;600\n" +
            "11;4;Tele2 Black;800\n" +
            "12;4;Tele2 Везде онлайн;450\n" +
            "13;5;Yota Интернет;720\n" +
            "14;5;Yota Безлимит;990\n" +
            "15;5;Yota Базовый;390\n",
            Encoding.UTF8);
    }
}

// =====

```

```

// TelecomOperator.cs
// =====

/// <summary>
/// Оператор связи (справочная таблица, сторона "один").
/// </summary>
public class TelecomOperator
{
    /// <summary>
    /// Идентификатор оператора связи.
    /// </summary>
    public int Id { get; set; }

    /// <summary>
    /// Название оператора связи.
    /// </summary>
    public string Name { get; set; }

    /// <summary>
    /// Конструктор с параметрами.
    /// </summary>
    public TelecomOperator(int id, string name)
    {
        Id = id;
        Name = name;
    }

    /// <summary>
    /// Конструктор по умолчанию.
    /// </summary>
    public TelecomOperator() : this(0, string.Empty)
    {
    }

    /// <summary>
    /// Возвращает строковое представление оператора связи.
    /// </summary>
    public override string ToString() => $"[{Id}] {Name}";
}

// =====
// Tariff.cs
// =====

/// <summary>
/// Тариф оператора связи (основная таблица, сторона "много").
/// </summary>
public class Tariff
{
    private int _priceMonth;

    /// <summary>
    /// Идентификатор тарифа.
    /// </summary>
    public int Id { get; set; }

    /// <summary>
    /// Идентификатор оператора связи (внешний ключ).
    /// </summary>
    public int OperatorId { get; set; }

    /// <summary>
    /// Название тарифа.
    /// </summary>
    public string Name { get; set; }

    /// <summary>
    /// Абонентская плата за месяц, руб./мес. Значение не может быть отрицательным.
    /// </summary>
    public int PriceMonth
    {
        get => _priceMonth;
        set
        {
            if (value < 0)
                throw new ArgumentException("Абонентская плата не может быть отрицательной.");

            _priceMonth = value;
        }
    }

    /// <summary>
    /// Конструктор с параметрами.
    /// </summary>
    public Tariff(int id, int operatorId, string name, int priceMonth)
    {
        Id = id;
        OperatorId = operatorId;
        Name = name;
        PriceMonth = priceMonth;
    }
}

```

```

    /// <summary>
    /// Конструктор по умолчанию.
    /// </summary>
    public Tariff() : this(0, 0, string.Empty, 0)
    {
    }

    /// <summary>
    /// Возвращает строковое представление тарифа.
    /// </summary>
    public override string ToString()
    => $"{Id} {Name}, оператор #{OperatorId}, абонентская плата: {PriceMonth} руб./мес.";
}

// =====
// QueryResult.cs
// =====

/// <summary>
/// Результат SQL-запроса: заголовки столбцов и строки со значениями.
/// </summary>
/// <param name="Headers">Имена столбцов.</param>
/// <param name="Rows">Строки результата.</param>
public record QueryResult(string[] Headers, List<string[]> Rows);

// =====
// DatabaseManager.cs
// =====

/// <summary>
/// Класс для инкапсуляции всей работы с SQLite.
/// </summary>
public class DatabaseManager
{
    private readonly string _connectionString;

    /// <summary>
    /// Создает объект менеджера БД. Если таблиц нет, они создаются.
    /// </summary>
    public DatabaseManager(string databasePath)
    {
        _connectionString = $"Data Source={databasePath}";
        CreateTables();
    }

    /// <summary>
    /// Импортирует тестовые данные из CSV-файлов.
    /// </summary>
    public void ImportFromCsv(string operatorsCsvPath, string tariffsCsvPath)
    {
        using var connection = OpenConnection();
        using var transaction = connection.BeginTransaction();

        var clearTariffs = connection.CreateCommand();
        clearTariffs.Transaction = transaction;
        clearTariffs.CommandText = "DELETE FROM tariffs;";
        clearTariffs.ExecuteNonQuery();

        var clearOperators = connection.CreateCommand();
        clearOperators.Transaction = transaction;
        clearOperators.CommandText = "DELETE FROM operators;";
        clearOperators.ExecuteNonQuery();

        string[] operatorLines = File.ReadAllLines(operatorsCsvPath);
        for (int i = 1; i < operatorLines.Length; i++)
        {
            if (string.IsNullOrEmpty(operatorLines[i]))
                continue;

            string[] parts = operatorLines[i].Split(';');
            if (parts.Length < 2)
                continue;

            var command = connection.CreateCommand();
            command.Transaction = transaction;
            command.CommandText = @"
                INSERT INTO operators (operator_id, operator_name)
                VALUES (@id, @name);";
            command.Parameters.AddWithValue("@id", int.Parse(parts[0], CultureInfo.InvariantCulture));
            command.Parameters.AddWithValue("@name", parts[1]);
            command.ExecuteNonQuery();
        }

        string[] tariffLines = File.ReadAllLines(tariffsCsvPath);
        for (int i = 1; i < tariffLines.Length; i++)
        {
            if (string.IsNullOrEmpty(tariffLines[i]))
                continue;

            string[] parts = tariffLines[i].Split(';');
            if (parts.Length < 4)

```

```

        continue;

        var command = connection.CreateCommand();
        command.Transaction = transaction;
        command.CommandText = @"
            INSERT INTO tariffs (tariff_id, operator_id, tariff_name, price_month)
            VALUES (@id, @operatorId, @name, @priceMonth);";
        command.Parameters.AddWithValue("@id", int.Parse(parts[0], CultureInfo.InvariantCulture));
        command.Parameters.AddWithValue("@operatorId", int.Parse(parts[1], CultureInfo.InvariantCulture));
        command.Parameters.AddWithValue("@name", parts[2]);
        command.Parameters.AddWithValue("@priceMonth", int.Parse(parts[3], CultureInfo.InvariantCulture));
        command.ExecuteNonQuery();
    }

    transaction.Commit();
}

/// <summary>
/// Возвращает все записи справочника операторов связи.
/// </summary>
public List<TelecomOperator> GetAllOperators()
{
    List<TelecomOperator> result = new();

    using var connection = OpenConnection();
    var command = connection.CreateCommand();
    command.CommandText = @"
        SELECT operator_id, operator_name
        FROM operators
        ORDER BY operator_id;";

    using var reader = command.ExecuteReader();
    while (reader.Read())
        result.Add(new TelecomOperator(reader.GetInt32(0), reader.GetString(1)));

    return result;
}

/// <summary>
/// Возвращает все тарифы.
/// </summary>
public List<Tariff> GetAllTariffs()
{
    List<Tariff> result = new();

    using var connection = OpenConnection();
    var command = connection.CreateCommand();
    command.CommandText = @"
        SELECT tariff_id, operator_id, tariff_name, price_month
        FROM tariffs
        ORDER BY tariff_id;";

    using var reader = command.ExecuteReader();
    while (reader.Read())
    {
        result.Add(new Tariff(
            reader.GetInt32(0),
            reader.GetInt32(1),
            reader.GetString(2),
            reader.GetInt32(3)));
    }

    return result;
}

/// <summary>
/// Возвращает тариф по идентификатору или null, если тариф не найден.
/// </summary>
public Tariff? GetTariffById(int id)
{
    using var connection = OpenConnection();
    var command = connection.CreateCommand();
    command.CommandText = @"
        SELECT tariff_id, operator_id, tariff_name, price_month
        FROM tariffs
        WHERE tariff_id = @id;";
    command.Parameters.AddWithValue("@id", id);

    using var reader = command.ExecuteReader();
    if (!reader.Read())
        return null;

    return new Tariff(
        reader.GetInt32(0),
        reader.GetInt32(1),
        reader.GetString(2),
        reader.GetInt32(3));
}

/// <summary>
/// Возвращает следующий свободный идентификатор тарифа.

```

```

/// </summary>
public int GetNextTariffId()
{
    using var connection = OpenConnection();
    var command = connection.CreateCommand();
    command.CommandText = "SELECT COALESCE(MAX(tariff_id), 0) + 1 FROM tariffs;";
    return Convert.ToInt32(command.ExecuteScalar(), CultureInfo.InvariantCulture);
}

/// <summary>
/// Добавляет новый тариф.
/// </summary>
public void AddTariff(Tariff tariff)
{
    using var connection = OpenConnection();
    var command = connection.CreateCommand();
    command.CommandText = @"
        INSERT INTO tariffs (tariff_id, operator_id, tariff_name, price_month)
        VALUES (@id, @operatorId, @name, @priceMonth);";
    AddTariffParameters(command, tariff);
    command.ExecuteNonQuery();
}

/// <summary>
/// Обновляет существующий тариф по Id.
/// </summary>
public void UpdateTariff(Tariff tariff)
{
    using var connection = OpenConnection();
    var command = connection.CreateCommand();
    command.CommandText = @"
        UPDATE tariffs
        SET operator_id = @operatorId,
            tariff_name = @name,
            price_month = @priceMonth
        WHERE tariff_id = @id;";
    AddTariffParameters(command, tariff);
    command.ExecuteNonQuery();
}

/// <summary>
/// Удаляет тариф по Id.
/// </summary>
public void DeleteTariff(int id)
{
    using var connection = OpenConnection();
    var command = connection.CreateCommand();
    command.CommandText = "DELETE FROM tariffs WHERE tariff_id = @id;";
    command.Parameters.AddWithValue("@id", id);
    command.ExecuteNonQuery();
}

/// <summary>
/// Возвращает тарифы выбранного оператора связи.
/// </summary>
public List<Tariff> GetTariffsByOperator(int operatorId)
{
    List<Tariff> result = new();

    using var connection = OpenConnection();
    var command = connection.CreateCommand();
    command.CommandText = @"
        SELECT tariff_id, operator_id, tariff_name, price_month
        FROM tariffs
        WHERE operator_id = @operatorId
        ORDER BY tariff_name;";
    command.Parameters.AddWithValue("@operatorId", operatorId);

    using var reader = command.ExecuteReader();
    while (reader.Read())
    {
        result.Add(new Tariff(
            reader.GetInt32(0),
            reader.GetInt32(1),
            reader.GetString(2),
            reader.GetInt32(3)));
    }

    return result;
}

/// <summary>
/// Выполняет SQL-запрос и возвращает универсальный табличный результат.
/// </summary>
public QueryResult ExecuteQuery(string sql)
{
    using var connection = OpenConnection();
    var command = connection.CreateCommand();
    command.CommandText = sql;

    using var reader = command.ExecuteReader();

```

```

        string[] headers = new string[reader.FieldCount];
        for (int i = 0; i < reader.FieldCount; i++)
            headers[i] = reader.GetName(i);

        List<string[]> rows = new();
        while (reader.Read())
        {
            string[] row = new string[reader.FieldCount];
            for (int i = 0; i < reader.FieldCount; i++)
                row[i] = reader.GetValue(i)?.ToString() ?? string.Empty;
            rows.Add(row);
        }

        return new QueryResult(headers, rows);
    }

    private void CreateTables()
    {
        using var connection = OpenConnection();

        var createOperators = connection.CreateCommand();
        createOperators.CommandText = @"
            CREATE TABLE IF NOT EXISTS operators (
                operator_id INTEGER PRIMARY KEY,
                operator_name TEXT NOT NULL
            );";
        createOperators.ExecuteNonQuery();

        var createTariffs = connection.CreateCommand();
        createTariffs.CommandText = @"
            CREATE TABLE IF NOT EXISTS tariffs (
                tariff_id INTEGER PRIMARY KEY,
                operator_id INTEGER NOT NULL,
                tariff_name TEXT NOT NULL,
                price_month INTEGER NOT NULL CHECK (price_month >= 0),
                FOREIGN KEY (operator_id) REFERENCES operators(operator_id)
                    ON UPDATE CASCADE
                    ON DELETE RESTRICT
            );";
        createTariffs.ExecuteNonQuery();
    }

    private SqliteConnection OpenConnection()
    {
        var connection = new SqliteConnection(_connectionString);
        connection.Open();

        var command = connection.CreateCommand();
        command.CommandText = "PRAGMA foreign_keys = ON;";
        command.ExecuteNonQuery();

        return connection;
    }

    private static void AddTariffParameters(SqliteCommand command, Tariff tariff)
    {
        command.Parameters.AddWithValue("@id", tariff.Id);
        command.Parameters.AddWithValue("@operatorId", tariff.OperatorId);
        command.Parameters.AddWithValue("@name", tariff.Name);
        command.Parameters.AddWithValue("@priceMonth", tariff.PriceMonth);
    }
}

// =====
// ReportBuilder.cs
// =====

/// <summary>
/// Построитель текстовых отчетов с использованием паттерна Fluent Interface.
/// </summary>
public class ReportBuilder
{
    private readonly DatabaseManager _db;
    private string _sql = string.Empty;
    private string _title = string.Empty;
    private string[] _headers = Array.Empty<string>();
    private int[] _widths = Array.Empty<int>();
    private bool _numbered;
    private string _footer = string.Empty;

    /// <summary>
    /// Конструктор принимает DatabaseManager для доступа к данным.
    /// </summary>
    public ReportBuilder(DatabaseManager db)
    {
        _db = db;
    }

    /// <summary>
    /// Задает SQL-запрос отчета.
    /// </summary>

```



```

public ReportBuilder Query(string sql)
{
    _sql = sql;
    return this;
}

/// <summary>
/// Задаёт заголовок отчета.
/// </summary>
public ReportBuilder Title(string text)
{
    _title = text;
    return this;
}

/// <summary>
/// Задаёт заголовки колонок отчета.
/// </summary>
public ReportBuilder Header(params string[] columns)
{
    _headers = columns;
    return this;
}

/// <summary>
/// Задаёт ширины колонок отчета.
/// </summary>
public ReportBuilder ColumnWidths(params int[] widths)
{
    _widths = widths;
    return this;
}

/// <summary>
/// Включает нумерацию строк отчета.
/// </summary>
public ReportBuilder Numbered()
{
    _numbered = true;
    return this;
}

/// <summary>
/// Добавляет итоговую строку в конец отчета.
/// В тексте можно использовать маркер {count}, который заменяется количеством строк.
/// </summary>
public ReportBuilder Footer(string label)
{
    _footer = label;
    return this;
}

/// <summary>
/// Формирует отчет и возвращает его как строку.
/// </summary>
public string Build()
{
    if (string.IsNullOrEmpty(_sql))
        throw new InvalidOperationException("SQL-запрос отчета не задан.");

    QueryResult result = _db.ExecuteQuery(_sql);
    StringBuilder sb = new();

    if (!string.IsNullOrEmpty(_title))
    {
        sb.AppendLine($"=== {_title} ===");
    }

    AppendHeader(sb);
    AppendSeparator(sb);

    for (int i = 0; i < result.Rows.Count; i++)
    {
        if (_numbered)
            AppendCell(sb, (i + 1).ToString(), GetWidth(0, 5));

        for (int c = 0; c < result.Rows[i].Length; c++)
        {
            int widthIndex = _numbered ? c + 1 : c;
            AppendCell(sb, result.Rows[i][c], GetWidth(widthIndex, 18));
        }

        sb.AppendLine();
    }

    if (!string.IsNullOrEmpty(_footer))
    {
        AppendSeparator(sb);
        sb.AppendLine(_footer.Replace("{count}", result.Rows.Count.ToString()));
    }

    return sb.ToString();
}

```

```

    }

    /// <summary>
    /// Выводит сформированный отчет в консоль.
    /// </summary>
    public void Print()
    {
        Console.WriteLine(Build());
    }

    /// <summary>
    /// Сохраняет отчет в текстовый файл.
    /// </summary>
    public void SaveToFile(string path)
    {
        File.WriteAllText(path, Build(), Encoding.UTF8);
    }

    private void AppendHeader(StringBuilder sb)
    {
        if (_numbered)
            AppendCell(sb, "№", GetWidth(0, 5));

        for (int i = 0; i < _headers.Length; i++)
        {
            int widthIndex = _numbered ? i + 1 : i;
            AppendCell(sb, _headers[i], GetWidth(widthIndex, 18));
        }

        sb.AppendLine();
    }

    private void AppendSeparator(StringBuilder sb)
    {
        int totalWidth = 0;
        int columnCount = _headers.Length + (_numbered ? 1 : 0);

        for (int i = 0; i < columnCount; i++)
            totalWidth += GetWidth(i, 18);

        sb.AppendLine(new string('-', totalWidth));
    }

    private int GetWidth(int index, int defaultWidth)
    {
        if (index >= 0 && index < _widths.Length)
            return _widths[index];

        return defaultWidth;
    }

    private static void AppendCell(StringBuilder sb, string value, int width)
    {
        string safeValue = value ?? string.Empty;
        if (safeValue.Length > width - 1)
            safeValue = safeValue.Substring(0, Math.Max(0, width - 2)) + "...";

        sb.Append($"{safeValue,-width}");
    }
}

```